

```

#!/usr/bin/env bash
#
=====
=====
# Qualcomm Hexagon NPU Multi-Device Process Domain (PD) Execution Wrapper
# Script: run_split_htp_model.sh
# Location: novaexopia/aesc/run_split_htp_model.sh
# Purpose: Offloads large models (>3.5GB) across virtual HTP domains (HTP0, HTP1)
# Grounded in: 7.md & llama.cpp Snapdragon Hexagon Developer Documentation
#
=====
=====

set -euo pipefail

# -----
# 1. Environment & Default Configuration
# -----
MODEL_NAME="${1:-qwen3.5-9b-q4_0.gguf}"
PROMPT="${2:-Hello, system operational check.}"
DEVICE_DOMAINS="HTP0,HTP1"
MAX_PD_RAM_MB=3584 # Hard 3.5GB per Process Domain ceiling
MIN_HOST_RAM_AVAIL_MB=4500 # Ensure phone has sufficient active RAM

MODELS_DIR="${HOME}/models"
RUNTIME_DIR="${HOME}/runtimes/snapdragon-hexagon"
MODEL_PATH="${MODELS_DIR}/${MODEL_NAME}"

echo "[*] Qualcomm Hexagon NPU Multi-Device Split Runner Initialized"
echo "  Model: ${MODEL_PATH}"
echo "  Devices: ${DEVICE_DOMAINS} (Split across 2 Process Domains)"
echo "  PD Max: ${MAX_PD_RAM_MB} MB / domain"

# -----
# 2. Pre-flight Validation: Model Existence & System RAM Check
# -----
if [[ ! -f "${MODEL_PATH}" ]]; then
  # Fallback check in current working directory
  if [[ -f "${MODEL_NAME}" ]]; then
    MODEL_PATH="${MODEL_NAME}"
  else
    echo "[-] ERROR: Model weights not found at ${MODEL_PATH}" >&2
    echo "  Please verify model GGUF file location." >&2
  fi
fi

```

```

        exit 1
    fi
fi

# Check available memory via /proc/meminfo
if [[ -f /proc/meminfo ]]; then
    MEM_AVAIL_KB=$(grep MemAvailable /proc/meminfo | awk '{print $2}')
    MEM_AVAIL_MB=$(( MEM_AVAIL_KB / 1024 ))
    echo "[+] Available System RAM: ${MEM_AVAIL_MB} MB"

    if (( MEM_AVAIL_MB < MIN_HOST_RAM_AVAIL_MB )); then
        echo "[!] WARNING: System available RAM (${MEM_AVAIL_MB} MB) is below
recommended threshold (${MIN_HOST_RAM_AVAIL_MB} MB)." >&2
        echo "    Triggering Android LMK cache flush / drop_caches if permitted..." >&2
        sync || true
    fi
else
    echo "[!] /proc/meminfo unavailable; skipping RAM pre-flight."
fi

# -----
# 3. Execution Parameter Setup
# -----
# Configure Hexagon driver environment flags
export
ADSP_LIBRARY_PATH="${RUNTIME_DIR}/lib;${ADSP_LIBRARY_PATH:-/vendor/lib/rfsa/adsp
}"
export LD_LIBRARY_PATH="${RUNTIME_DIR}/lib:${LD_LIBRARY_PATH:-}"

# Enforce multi-device allocation string: HTP0 for layers 0-24, HTP1 for layers 25-48
export M="${MODEL_PATH}"
export D="${DEVICE_DOMAINS}"

echo "[+] Offloading layers across HTP0 (PD 1: ~2.8GB) and HTP1 (PD 2: ~2.7GB)..."
echo "[+] Hardware RPC tensor handoffs active between Process Domains."

# -----
# 4. Launch Inference via Snapdragon Hexagon Backend
# -----
if [[ -x "${RUNTIME_DIR}/run-completion.sh" ]]; then
    exec "${RUNTIME_DIR}/run-completion.sh" -p "${PROMPT}"
elif [[ -x "./run-completion.sh" ]]; then
    exec ./run-completion.sh -p "${PROMPT}"

```

```
elif command -v llama-cli >/dev/null 2>&1; then
    exec llama-cli \
        -m "${MODEL_PATH}" \
        --device "${DEVICE_DOMAINS}" \
        -p "${PROMPT}" \
        -c 4096 \
        --temp 0.2 \
        --n-predict 512
else
    echo "[!] Neither run-completion.sh nor llama-cli found in active PATH."
    echo "[*] Standard invocation syntax for your daemon environment:"
    echo "    M=\"${MODEL_PATH}\" D=\"${DEVICE_DOMAINS}\" ./run-completion.sh -p
\"${PROMPT}\""
    exit 0
fi
```